pgAdmin 4 — Mozilla Firefox

pgAdmin 4 | Backlog · Kanban-BDR-2026 | como usar a2ps para gerar p | a2ps without margin (borde

http://127.0.0.1:5050/browser/

pgAdmin File Object Tools Help

Dashboard | Properties | SQL | Statistics | schema.sql

clima_alerta/postgres@Localhost

No limit

Query | Query History

```
-- Remoção das tabelas existentes
DROP TABLE IF EXISTS Alerta      CASCADE;
DROP TABLE IF EXISTS Relato      CASCADE;
DROP TABLE IF EXISTS Evento      CASCADE;
DROP TABLE IF EXISTS TipoEvento  CASCADE;
DROP TABLE IF EXISTS Localizacao CASCADE;
DROP TABLE IF EXISTS Usuario     CASCADE;
DROP TABLE IF EXISTS CategoriaUsuario CASCADE;

-- Criação das tabelas

-- 1. CategoriaUsuario
CREATE TABLE CategoriaUsuario (
    idCategoria SERIAL PRIMARY KEY,
    nome        VARCHAR(50) NOT NULL,
    nivel_acesso INT NOT NULL,
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 56 msec.
```

Total rows: | Query complete 00:00:00.056 | LF | Ln 77, Col 4

- clima_alerta
  - Casts
  - Catalogs
  - Event Triggers
  - Extensions
  - Foreign Data Wrappers
  - Languages
  - Publications
  - Schemas (1)
    - public
      - Aggregates
      - Collations
      - Domains
      - FTS Configurations
      - FTS Dictionaries
      - FTS Parsers
      - FTS Templates
      - Foreign Tables
      - Functions
      - Materialized Views
      - Operators
      - Procedures

13:25

pgAdmin 4 — Mozilla Firefox

pgAdmin 4 | Backlog · Kanban-BDR-2026

http://127.0.0.1:5050/browser/

File | Object | Tools | Help

Object Explorer

- Languages
- Publications
- Schemas (1)
  - public
    - Aggregates
    - Collations
    - Domains
    - FTS Configurations
    - FTS Dictionaries
    - FTS Parsers
    - FTS Templates
    - Foreign Tables
    - Functions
    - Materialized Views
    - Operators
    - Procedures
    - Sequences
    - Tables (7)
      - alerta
      - categoriausuario
      - evento
      - localizacao

Dashboard | Properties | SQL | Statistics | schema.sql

clima_alerta/postgres@Localhost

No limit

Query | Query History

```sql
-- 7. Alerta
CREATE TABLE Alerta (
    idAlerta SERIAL PRIMARY KEY,
    mensagem VARCHAR(50),
    dataHora TIMESTAMP,
    nivel    VARCHAR(30),
    idEvento INT REFERENCES Evento(idEvento)
);

-- SELECT * FROM Alerta;
-- SELECT * FROM Evento;
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 56 msec.
```

Total rows: | Query complete 00:00:00.056 | LF | Ln 77, Col 4

12:55

```
-- Remocao das tabelas existentes
DROP TABLE IF EXISTS Alerta      CASCADE;
DROP TABLE IF EXISTS Relato      CASCADE;
DROP TABLE IF EXISTS Evento      CASCADE;
DROP TABLE IF EXISTS TipoEvento  CASCADE;
DROP TABLE IF EXISTS Localizacao CASCADE;
DROP TABLE IF EXISTS Usuario     CASCADE;
DROP TABLE IF EXISTS CategoriaUsuario CASCADE;

-- Criacao das tabelas

-- 1. CategoriaUsuario
CREATE TABLE CategoriaUsuario (
    idCategoria SERIAL PRIMARY KEY,
    nome        VARCHAR(50) NOT NULL,
    nivel_acesso INT NOT NULL,
    descricao   VARCHAR(200),
    CONSTRAINT NomeCategoria UNIQUE (nome)
    -- Havera apenas um nome para cada tipo de categoria (ex.administrador, oper
ador, etc)
);

-- 2. Usuario
CREATE TABLE Usuario (
    idUsuario SERIAL PRIMARY KEY,
    nome      VARCHAR(50),
    email     VARCHAR(50),
    senhaHash TEXT NOT NULL,
    idCategoria INT REFERENCES CategoriaUsuario(idCategoria)
);

-- 3. Localizacao
CREATE TABLE Localizacao (
    idLocalizacao SERIAL PRIMARY KEY,
    latitude      DECIMAL(5,3),
    longitude     DECIMAL(5,3),
    cidade        VARCHAR(50),
    estado        CHAR(2)
);

-- 4. TipoEvento
CREATE TABLE TipoEvento (
    idTipoEvento SERIAL PRIMARY KEY,
    nome         VARCHAR(30),
    descricao    VARCHAR(50)
);

-- 5. Evento
CREATE TABLE Evento (
    idEvento      SERIAL PRIMARY KEY,
    titulo        VARCHAR(50),
    descricao     VARCHAR(50),
    dataHora      TIMESTAMP,
    status        VARCHAR(10),
    idTipoEvento  INT REFERENCES TipoEvento(idTipoEvento),
    idLocalizacao INT REFERENCES Localizacao(idLocalizacao)
);

-- 6. Relato
CREATE TABLE Relato (
    idRelato  SERIAL PRIMARY KEY,
    texto     VARCHAR(50),
    dataHora  TIMESTAMP,
    idEvento  INT REFERENCES Evento(idEvento),
    idUsuario INT REFERENCES Usuario(idUsuario)
);

-- 7. Alerta
CREATE TABLE Alerta (
    idAlerta SERIAL PRIMARY KEY,
    mensagem VARCHAR(50),
```

```
    dataHora TIMESTAMP,
    nivel    VARCHAR(30),
    idEvento INT REFERENCES Evento(idEvento)
);

-- SELECT * FROM Alerta;
-- SELECT * FROM Evento;
-- SELECT * FROM TipoEvento;
-- SELECT * FROM Localizacao;
-- SELECT * FROM Usuario;
-- SELECT * FROM Relato;
-- SELECT * FROM CategoriaUsuario;
```